FITVISION AI

AI-Powered Virtual Try-On & Personal Fashion Assistant

Project purpose: Build an Android application that combines personalized style profiling, digital wardrobe management, clothing discovery, outfit recommendations, and AI-powered 2D virtual try-on.

Core idea

Connect what I already own, what I want to buy, and how it looks on me - before I actually wear or purchase it.

Project type	Android + AI + Computer Vision + Cloud Backend
Team	5 diploma Computer Engineering students
Target timeline	20 days for first working prototype
Primary output	2D AI-generated virtual try-on image
Future scope	3D avatar and 360-degree try-on

Prepared as a project proposal / prototype blueprint

Contents

This document consolidates the complete project discussion into a single implementation and presentation reference.

1. Project Vision and Problem Statement

FitVision AI is an Android-based personal fashion assistant. A user creates a visual profile using a few personal images, can add clothes they already own, can explore a catalog of clothes inside the app, can request style recommendations, and can virtually try selected garments using AI-generated images.

The core problem is uncertainty. Users often do not know how a garment or complete outfit may look on them before wearing it or purchasing it online. The project aims to reduce that uncertainty with a mobile-first visual assistant.

The project is not presented as the invention of virtual try-on. Existing systems already demonstrate virtual try-on, digital wardrobes, or fashion recommendations. The academic contribution is the integration of these functions into one explainable Android workflow, together with a custom catalog, personal wardrobe, occasion-based styling, and a try-before-buy experience.

2. Objectives and Scope

- Develop a practical Android fashion application with a modern user interface.
- Create a personalized style profile from user-provided images and preferences.
- Allow users to store their existing clothes in a digital wardrobe.
- Provide a built-in clothing catalog for discovery and potential purchase decisions.
- Generate personalized clothing and outfit recommendations.
- Provide AI-powered 2D virtual try-on for selected garments.
- Allow users to save, revisit, compare, and manage generated looks.
- Demonstrate practical integration of Android, cloud services, computer vision, generative AI, and recommendation logic.

Primary scope for 20 days: multi-view profile analysis, 2D virtual try-on, clothing catalog, wardrobe, recommendations, saved looks, and a clean Android experience. True 3D/360-degree reconstruction is intentionally future scope.

3. Complete User Experience

Step	Action
1	Register / Login
2	Upload front, left and right images
3	Choose preferences / occasion
4	AI creates a style profile
5	Open Home / Shop / Wardrobe / AI Stylist
6	Select a garment or complete outfit
7	Generate virtual try-on
8	View, save, share or replace the look
9	Return later through history / wishlist

Example: a user selects “Professional”, browses a white shirt, sees a compatibility score, taps Try On, receives a generated preview, then completes the look with trousers and shoes recommended by the system.

4. Core Features

1. User profile - Front / left / right image upload; preferences; editable style profile.

2. Style analysis - Visual features for recommendation purposes: face shape category, posture/body profile, color palette and preferences.

3. My Wardrobe - Upload and categorize clothes already owned by the user.

4. Shop catalog - Built-in catalog of shirts, T-shirts, trousers, jeans, jackets, dresses, etc., with metadata and price.

5. Style categories - Casual, aesthetic, professional, formal, college, streetwear, party, night out, traditional and sporty.

6. Occasion styling - Generate recommendations for presentations, college, parties, events, etc.

7. Recommendation score - Explainable score based on style, color, occasion, user preference and outfit compatibility.

8. Virtual try-on - Generate a 2D image of the user wearing a selected garment or outfit.

9. Complete look generator - Combine top, bottom and footwear into a full outfit preview.

10. AI Stylist - Natural-language requests such as “What should I wear for college?”

11. Wishlist - Save products to revisit later.

12. Try-on history - Reopen or delete generated looks.

13. Dark mode / settings - Theme, preferences and account controls.

5. AI and Computer Vision

The project contains two distinct AI areas: (1) profile / style analysis and recommendation, and (2) virtual try-on image generation. Keeping them separate reduces project risk.

User image analysis can use mobile-friendly computer vision for face/pose information and server-side processing for heavier operations. The recommendation layer should initially be rule/scoring based rather than a custom trained ML model.

Possible signals include face-shape category, pose/posture, approximate proportions, color palette, user-selected style, occasion, garment color, garment category, and compatibility with existing wardrobe items. These are style inputs, not medical or exact body measurements.

Recommendation philosophy: the app should explain recommendations rather than presenting arbitrary “beauty” judgments. Example: “Recommended because it matches your selected professional style and works with the colors in your wardrobe.”

6. Virtual Try-On Strategy

Virtual try-on is the technically hardest component. The team should not train a large generative model from scratch in a 20-day diploma project. Instead, evaluate an existing research VTON implementation and integrate it behind a backend API.

Candidate model families discussed: CatVTON and IDM-VTON. Both are research/open-source VTON projects intended to generate a person wearing a reference garment. Model choice should be based on image quality, compatibility, GPU availability, speed, and licensing.

Proof-of-concept milestone: before investing heavily in UI, demonstrate one successful pipeline: person image + garment image -> acceptable generated try-on image. If the chosen model cannot run in the available environment, change the model early.

Architecture principle: the Android phone handles UI and uploads; the AI server handles heavy PyTorch inference on suitable GPU infrastructure.

7. Clothing Catalog and Personal Wardrobe

The application has two clothing sources. My Wardrobe contains clothes the user already owns. Shop contains the project's in-app catalog of clothes the user may consider buying.

Field	Purpose
productId	Unique product identifier
name	Display name
imageURL	Clothing image
category	Shirt / T-shirt / Jeans / etc.
color	Primary color
style	Casual / formal / streetwear / etc.
occasion	College / party / professional / etc.
price	Prototype price
brand	Optional brand label

A prototype catalog of roughly 50-100 well-tagged products is sufficient for a convincing demo. The goal is not to reproduce a full e-commerce inventory.

8. Recommendation Engine

For the initial version, use an explainable scoring formula rather than training a complex recommendation model.

```
Recommendation Score = Color Compatibility + Style Compatibility + Occasion Compatibility + User Preference + Wardrobe Compatibility
```

The exact weights can be tuned experimentally. The app can normalize the result to a simple 0-100 compatibility score for presentation.

Signal	Example	Weight idea
Color	Black top with user-selected palette	25%
Style	Professional garment for professional mode	25%
Occasion	Suitable for presentation	20%
Preference	Matches user-selected style	20%
Wardrobe	Pairs with existing bottom/shoes	10%

9. AI Stylist and Shopping Flow

The AI Stylist provides a conversational entry point to the recommendation system. A user can ask for an occasion or style, and the system returns a small, understandable set of options.

```
User: "I have a college presentation tomorrow."

AI Stylist: "Try a white shirt with black trousers and clean sneakers for a simple professional look."

[ TRY THIS LOOK ]
```

The app can then launch the recommendation and virtual try-on flow. This creates a strong demo because the natural-language request becomes a visual result.

10. Android Application

Recommended mobile stack: Kotlin + Android Studio + XML/Material Design for a 20-day student build. Jetpack Compose is possible, but XML may be faster if the team already knows traditional Android development.

Screen	Main content
Splash	Logo + tagline
Login / Register	Authentication
Create Profile	Front / left / right image upload
Analysis	Progress / profile creation
Home	Recommendations, wardrobe, shop, AI stylist
Shop	Clothing catalog + filters
Wardrobe	User-owned clothes
Try-On	Person + garment selection
Result	Generated image + save/share/try again
Recommendations	Ranked clothes / looks
Profile / Settings	Preferences, dark mode, delete data

11. Backend and API

Use Python + FastAPI as the main backend. This is convenient because the VTON model and other heavy AI components are also Python/PyTorch based.

Method	Endpoint	Purpose
POST	/auth/register	Create account / link user
POST	/profile/upload	Upload profile image references
POST	/profile/analyze	Create style profile
GET	/products	Browse catalog
GET	/products/{id}	Product detail
POST	/wardrobe	Add wardrobe item
GET	/wardrobe	List wardrobe
POST	/recommendations	Generate ranked suggestions
POST	/tryon	Create try-on job
GET	/tryon/{id}	Fetch job/result
POST	/wishlist	Save product
DELETE	/wishlist/{id}	Remove product

12. Database and Storage

Use Firebase Authentication for accounts, Firestore for structured data, and Firebase Storage for images. Firestore should store metadata and references rather than large image binaries.

```
users/{userId}
name, email, profile, preferences, createdAt

products/{productId}
name, imageURL, category, color, style, occasion, price, brand

wardrobe/{itemId}
userId, imageURL, category, color, style

tryon_history/{tryonId}
userId, garmentId, resultURL, timestamp

wishlist/{id}
userId, productId
```

13. Complete Technology Stack

Layer	Technology	Purpose
Mobile	Android + Kotlin	Application
UI	XML + Material Design	Screens and interaction
Networking	Retrofit + OkHttp	REST communication
Async	Kotlin Coroutines	Background/network work
Backend	Python + FastAPI	REST API + orchestration
AI	PyTorch	VTON and AI workloads
VTON	CatVTON / IDM-VTON	2D virtual try-on
Computer Vision	OpenCV	Image processing
Face Analysis	ML Kit / equivalent	Face-related visual features
Pose	MediaPipe / equivalent	Pose / landmark analysis
Database	Firebase Firestore	Structured data
Auth	Firebase Authentication	Accounts
Storage	Firebase Storage	Images
Git	Git + GitHub	Version control

14. System Architecture

```
ANDROID APP
| HTTPS / REST
v
FASTAPI BACKEND
|--------------------|
v v
FIREBASE AI GPU SERVER
Firestore PyTorch
Storage CatVTON / IDM-VTON
Auth Image Processing
| |
| v
| Generated Image
|--------------------|
```

v
Android Result Screen

This separation keeps the mobile application lightweight while allowing the heavy AI workload to run on a suitable GPU environment.

15. Modules and Team Responsibilities

Member	Primary responsibility	Typical deliverables
1 - Android	UI, navigation, auth screens, shop, try-on result	Android screens, Retrofit integration, Material UI
2 - Backend	FastAPI, Firebase, REST APIs	Endpoints, Firestore/Storage integration
3 - AI/CV	Image analysis, style profile, recommendation logic	Preprocessing, visual signals, scoring
4 - VTON	Model setup, GPU inference, output handling	CatVTON/IDM-VTON service and API
5 - Integration	Catalog, wardrobe, wishlist, history, testing	Data model, end-to-end integration, QA

All five members should contribute to testing, documentation, diagrams, and final demo preparation. No member should be isolated to documentation only.

16. 20-Day Development Plan

Days	Focus	Exit condition
1-2	GitHub, Android skeleton, Firebase, FastAPI, AI environment	All environments run
3-5	Login, Home, navigation; VTON proof-of-concept in parallel	One person+garment result
6-8	Profile upload, catalog, wardrobe, image storage	End-to-end data flow
9-11	Android -> FastAPI -> VTON integration	Try-on request works from app
12-14	Recommendations, styles, AI stylist, wishlist/history	Complete recommendation loop
15-17	UI polish, dark mode, loading/error states	Demo-ready UI
18-19	Functional and edge-case testing	Major bugs fixed
20	Freeze code, PPT, report, DFD/UML/ER, demo practice	Final submission

Critical rule: do not spend the first half of the schedule polishing UI before the VTON proof-of-concept is working.

17. Testing Strategy

- Image tests: clear, dark, blurry, different backgrounds, multiple poses and body orientations.
- Clothing tests: shirts, T-shirts, jackets, different colors and styles.
- API tests: valid requests, missing fields, invalid image type, timeout, AI failure.
- UI tests: login, upload, navigation, save/delete, history, wishlist, retry.
- Performance tests: image size, loading time, network failure, repeated try-on jobs.
- User tests: ask a few classmates to complete a full try-on flow and report usability issues.

18. Privacy, Security and Ethics

- Clearly tell users that personal images are being processed for profile and try-on purposes.
- Allow users to delete uploaded profile images and generated results.
- Use Firebase security rules and authenticated access to user data.
- Avoid presenting AI style scores as objective measurements of attractiveness or health.
- Describe results as visualizations and recommendations, not guarantees of real-world garment fit.
- Before any commercial release, review model licenses, data retention practices and third-party API terms.

19. Limitations and Risk Management

Risk	Impact	Mitigation
VTON model fails / slow	Very high	Prove model on Day 1-3; keep a fallback model or API plan
No suitable GPU	High	Reserve cloud/college GPU early; test inference before UI build
Poor user images	Medium	Image quality checks, cropping guidance, retry flow
Generated image not realistic	High	Use good preprocessing, realistic demo claims, compare models
20-day scope creep	High	Freeze MVP; treat 3D as future scope
Privacy / permissions	High	Authentication, secure rules, delete controls
Model license mismatch	High	Check license before publication/deployment

20. Existing Solutions and Project Differentiation

The project concept overlaps with several real-world products and research systems. Examples discussed include Google's Doppl, Google Shopping virtual try-on, YouCam, Whering, Style DNA and research systems such as KiseKloset. Their existence is not a blocker; it means the project needs a clear scope and differentiator.

Existing direction	Strong at	Your project angle
Google / Doppl-style VTO	Virtual try-on and discovery	Android-first educational prototype + wardrobe + recommendation + explainability
Google Shopping VTO	Try-on from shopping listings	Integrated personal wardrobe + occasion styling + complete-look generation
YouCam	Wardrobe / beauty / styling	Single student project combining wardrobe, shopping and try-on
Whering	Digital wardrobe / outfits	Add generative try-on and explainable recommendations
Style DNA-type apps	Personal style recommendations	Combine profile + catalog + wardrobe + visual try-on
Research VTO systems	Generation quality / methodology	End-to-end product-style Android workflow

Recommended faculty answer: "We are not claiming to invent virtual try-on technology. Existing systems already provide parts of this experience. Our project integrates personalized style profiling, a digital wardrobe, an in-app catalog, occasion-based recommendations, explainable scoring, complete outfit generation, and AI virtual try-on into one Android workflow as an academic prototype."

21. Viva / Faculty Questions

Q: Why Android?

A: The target user experience is mobile-first, and modern Android phones already provide the camera, storage and interaction capabilities needed for a fashion assistant.

Q: Why not run the VTON model on the phone?

A: Large generative VTON models are GPU-heavy. Running them behind a server API keeps the mobile app lightweight and allows the team to use suitable GPU infrastructure.

Q: Why three photos?

A: Multiple views can improve the style/profile representation, while the first version still uses a 2D try-on model for the final result.

Q: Why not 3D?

A: True 3D body reconstruction and cloth simulation are higher-risk than a 20-day student prototype. 3D/360 is kept as future scope.

Q: What is your unique contribution?

A: The integrated Android workflow: profile + wardrobe + catalog + recommendation + complete outfit + virtual try-on + explainable scoring.

Q: What if AI recommends badly?

A: Recommendations are advisory and use an explainable scoring system that can be tuned and overridden by the user.

Q: How is the price used?

A: Prototype products include price so the user can compare a visualized look with a potential purchase. Actual e-commerce checkout is future scope.

Q: What is your hardest module?

A: The VTON inference pipeline. Therefore it is validated first, before the app is fully built.

22. Minimum Viable Product

If the team runs short on time, the project should still be considered successful when this core path works smoothly:

```
LOGIN
-> Upload person image
-> Browse / select garment
-> Virtual Try-On
-> Generated result
-> Save result
```

Everything else - three-view profile, wardrobe, recommendations, AI Stylist, wishlist, history, dark mode, complete-look generation - is an enhancement around this core demo.

23. Future Scope

- Full 3D user avatar and 360-degree garment visualization.
- AR try-on using the live phone camera.
- Improved body measurement estimation using computer vision.
- Voice-based AI stylist and hands-free outfit search.
- Weather-aware and location-aware clothing recommendations.
- Price comparison and direct e-commerce integration.

- Long-term preference learning from likes, saves and purchases.
- Expanded Indian/ethnic clothing categories and regional styling.
- Personalized packing suggestions for trips and events.

24. Final Project Summary

FitVision AI is an Android-based AI personal fashion assistant that lets users create a visual style profile, manage their own wardrobe, explore an in-app clothing catalog, receive personalized outfit recommendations, and generate 2D virtual try-on previews before wearing or buying clothes. The system combines Android development, computer vision, generative AI, cloud storage, backend APIs and explainable recommendation logic.

Tagline: “Try what you own. Explore what you want. See how it looks on you before you wear or buy it.”

Recommended project title for submission: “FitVision AI - An Android-Based AI Virtual Try-On and Personal Fashion Assistant”.

Reference Notes

- Google Blog - Doppl / AI fashion experimentation: https://blog.google/innovation-and-ai/models-and-research/google-labs/doppl/
- Google Blog India - Virtual apparel try-on in India: https://blog.google/intl/en-in/products/virtual-apparel-try-on-tool-comes-to-india/
- CatVTON project: https://github.com/Zheng-Chong/CatVTON
- IDM-VTON project: https://github.com/yisol/IDM-VTON
- CatVTON paper: https://arxiv.org/abs/2407.15886
- KiseKloset research: https://arxiv.org/abs/2506.23471

Note: model availability, hardware requirements, and software licenses can change. Re-check the official project repository and license before deployment or public distribution.